

UNIGIGS

Complete Project Report – 12 Main Headings
Student-Focused Micro-Task Freelancing Platform

Project: UNIGIGS

Prepared by: P Hemanth Kumar Reddy

Registration No.: 99230030737

Major: AI & ML

Institution: Kalasalingam Academy of Research and Education

A student-oriented digital marketplace designed to connect students with small businesses and clients for affordable, short-term services and practical work opportunities.

1. Project Overview

UNIGIGS is a student-focused micro-task freelancing platform that connects students who have useful skills with small businesses, individuals, and clients who need affordable short-term assistance. The platform is intended to create a practical bridge between academic learning and real-world work.

Students can create profiles, list their skills and services, discover suitable gigs, apply for opportunities, and manage their projects. Clients can post tasks, search for suitable student talent, review applications, and manage completed work. The platform is designed around simplicity, affordability, trust, and accessibility.

The main idea is to make small freelance opportunities easier to discover while allowing students to build experience, portfolios, professional confidence, and income. UNIGIGS can support tasks such as design, content creation, data entry, tutoring, social-media assistance, basic technology support, and other student-friendly services.

2. Problem Statement, Aim & Objectives

Many students have skills but struggle to find suitable entry-level work because conventional freelancing platforms can be highly competitive, require established portfolios, or are not specifically designed around student schedules. At the same time, small businesses often need affordable help for short-term tasks but may not have the budget to hire full-time professionals.

Aim: To develop a student-centered freelancing ecosystem that makes short-term work opportunities easier to find while helping small businesses access affordable and capable student talent.

Objectives:

- Create a simple registration and profile system for students and clients.
- Allow students to showcase skills, services, experience, and portfolio information.
- Enable clients to post gigs and describe task requirements clearly.
- Provide search and discovery features for finding relevant opportunities.
- Support smart matching between student skills and client requirements.
- Provide dashboards for managing applications, gigs, and project progress.
- Build a foundation for secure communication, reviews, and payments.
- Improve employability by giving students practical project experience.

3. Scope, Users & Use Cases

Scope: UNIGIGS focuses on small, manageable freelance or micro-task opportunities that can be completed by students. The system can initially operate as a web platform and later expand to mobile applications and more advanced marketplace services.

Primary Users:

- **Students/Freelancers:** create profiles, add skills, discover gigs, apply for work, manage opportunities, and build experience.
- **Clients/Small Businesses:** create requirements, publish gigs, review applicants, select suitable students, and manage projects.
- **Administrator:** support platform governance, user management, moderation, security, and system-level monitoring.

Typical Use Cases:

- Student registration → profile creation → skill listing → gig search → application → project completion.
- Client registration → gig creation → applicant review → student selection → project management → review.

- Administrator login → user/content monitoring → issue handling → platform maintenance.

4. Functional & Non-Functional Requirements

Functional Requirements:

- User registration and login.
- Student and client profile management.
- Skill and service listing.
- Gig/job creation and management.
- Search, filtering, and opportunity discovery.
- Student application and client selection workflow.
- Dashboard for managing active and completed work.
- Reviews and ratings foundation.
- Secure authentication using JWT-based access control.
- REST API communication between frontend and backend.

Non-Functional Requirements:

- Usability: the interface should be simple and understandable for first-time users.
- Performance: common actions should respond quickly under expected student-project workloads.
- Security: passwords and authentication data must be protected and access should be controlled.
- Scalability: the architecture should support future growth in users, gigs, and transactions.
- Reliability: data operations should be consistent and recoverable.
- Maintainability: modular frontend, backend, and database components should be easy to update.

5. Modules & Key Features

User Authentication

- Registration, login, logout, JWT-based authentication, and protected access.

Student Profile

- Personal information, skills, services, experience, portfolio details, and availability.

Client & Gig Management

- Client profile, gig posting, requirements, budget, status, and project management.

Search & Smart Matching

- Search by skills/category and future recommendation logic based on skill-to-task similarity.

Applications & Opportunities

- Students can discover suitable gigs and submit applications; clients can review candidates.

Dashboard

- Centralized view of active gigs, applications, projects, and progress.

Reviews & Trust

- Ratings/reviews and verification mechanisms planned to improve marketplace trust.

Payments & Communication

- Real-time chat and integrated secure payments are planned as extended capabilities.

6. System Architecture & Technology Stack

The current architecture follows a client-server approach. The React.js frontend provides the user interface, while the Node.js and Express.js backend exposes REST APIs and handles business logic. PostgreSQL stores structured application data. JWT is used for authentication and protected API access.

Technology Stack:

- **Frontend:** React.js
- **Backend:** Node.js with Express.js
- **API:** REST APIs
- **Database:** PostgreSQL
- **Authentication:** JSON Web Tokens (JWT) and protected password handling
- **Development Approach:** modular client-server architecture

High-Level Architecture:

- User → React.js interface → REST API → Express/Node.js business logic → PostgreSQL database.
- Authentication middleware validates JWT tokens before protected operations.
- The architecture allows future integration of payment gateways, chat services, recommendation engines, analytics, and mobile clients.

7. Methodology & Development Process

The project follows an iterative software-development process. Requirements are first identified and converted into system features. The architecture and data model are then designed before implementing the frontend and backend. Testing and user feedback are used to refine the prototype.

Development Stages:

- Requirement analysis
- System and interface design
- Frontend development
- Backend and REST API development
- Database implementation
- Authentication and security
- Testing and validation
- Deployment preparation
- Maintenance and future enhancement

8. System Flow, API & Database Design

System Flow:

- Register/Login → Authenticate user → Complete profile → Browse or post gigs → Search/match → Apply/select → Manage project → Complete task → Review/rating.
- Client-side actions are sent to backend REST endpoints.

- Backend validates requests, applies business rules, and reads/writes data in PostgreSQL.
- Protected operations require successful authentication.

Representative API Areas:

- Authentication APIs for registration and login.
- User/profile APIs for managing account and skill information.
- Gig APIs for creating, reading, updating, and managing opportunities.
- Application APIs for submitting and reviewing applications.
- Review APIs for feedback and ratings.

Core Database Entities:

- Users
- Profiles
- Skills
- Gigs
- Applications
- Projects
- Reviews/Ratings
- Payments (planned)

9. Security, Testing & Validation

Security:

- JWT-based authentication for protected resources.
- Password protection using secure password-handling practices.
- Role-based access considerations for students, clients, and administrators.
- Input validation and controlled API access.
- Future payment and sensitive operations should use trusted, secure third-party infrastructure.

Testing & Validation:

- Unit and component testing for important application functions.
- API testing for authentication, gig, profile, and application operations.
- Integration testing between frontend, backend, and database.
- Usability testing with representative student/client users.
- Prototype validation involved 10 users; 9 users gave positive responses such as liking or loving the prototype.
- Feedback highlighted the simple interface, student earning concept, clear job listings, and easy application process.
- Suggested improvements included chat, payments, trust/verification, and more detailed profiles.

10. Business Model, Market & Competition

Business Model:

- **Marketplace model:** revenue can be generated through a small service/platform fee on successful transactions.

- **Freemium model:** basic access can remain free while premium visibility, advanced tools, or business features may be offered later.

Market Opportunity:

- The platform targets the growing demand for flexible freelance work and the increasing need for affordable digital services.
- Planning estimates in the project material identify a large global freelance opportunity, an India-focused serviceable market, and a targeted early-stage obtainable market. These figures are strategic planning assumptions rather than audited market measurements.

Competition & Differentiation:

- Large freelance marketplaces provide broad reach but are not specifically optimized for students.
- UNIGIGS differentiates through student-focused onboarding, smaller task sizes, affordability, skill-building, local/college-oriented outreach, and a simpler entry path for beginners.

Go-To-Market:

- Instagram and other social-media promotion.
- WhatsApp communities and student networks.
- College partnerships, workshops, and campus events.
- Local business outreach and flyers/posters.
- SEO and student-focused content.
- Influencer and community collaborations.

11. Benefits, Roadmap, Risks & Future Scope

Benefits:

- Students gain practical experience, income opportunities, portfolios, and professional confidence.
- Small businesses receive access to affordable, flexible assistance.
- Colleges can benefit from stronger links between learning and real-world project work.
- The platform can encourage entrepreneurship and peer-to-business collaboration.

Roadmap & Future Scope:

- AI-powered job and freelancer recommendations.
- Real-time chat and notifications.
- Integrated secure payments and transaction management.
- Ratings, reviews, and verified profiles.
- Dedicated mobile application.
- Advanced analytics and dashboards.
- Fraud/risk monitoring and stronger trust mechanisms.
- Expansion through college and local-business partnerships.

Key Risks & Mitigation:

- Low initial adoption → campus partnerships and targeted onboarding.
- Trust and quality concerns → ratings, reviews, verification, and moderation.

- Payment/security risk → use secure authentication and trusted payment infrastructure.
- Marketplace imbalance → focus on both student acquisition and client acquisition.
- Competition → maintain a clear student-first positioning and simple user experience.

12. Conclusion & References

UNIGIGS proposes a focused digital marketplace that connects student talent with small, practical work opportunities. By combining profiles, skills, gig posting, search, matching, applications, dashboards, and a secure technical foundation, the platform can help students gain experience while providing businesses with accessible support.

The current architecture using React.js, Node.js, Express.js, REST APIs, PostgreSQL, and JWT authentication provides a practical foundation for a scalable student marketplace. Future additions such as AI recommendations, chat, payments, verification, analytics, and a mobile app can strengthen the platform and improve trust and usability.

References / Project Source Material:

- UNIGIGS project concept and planning material prepared for the academic project.
- UNIGIGS technology-stack, methodology, and system-flow diagrams.
- Prototype validation feedback collected from representative users.
- General software engineering principles for client-server systems, REST APIs, relational databases, authentication, testing, and marketplace design.